Joint Path Planning of Truck and Drone Delivery System Under Wind Disturbances

Sai Siddhartha Vivek Dhir Rangoju Mohit Jangid Dhruv Devatha Kuldip Gami Krish Santoki
Northeastern University, Boston, MA, USA
{rangoju.s, jangid.mo, devatha.d, gami.k, santoki.k}@northeastern.edu

***Abstract*—With the recent progress in autonomous drone delivery systems, we explore the problem of collaborative delivery with truck and drone, where a single package is delivered to a target location through their coordination. Unlike existing approaches, that treat routing and control as separate problems, we consider the effect of wind on drone flight, which significantly impacts delivery time and energy use. In this project, we address the joint problem of launch timing, wind-aware path planning, and trajectory tracking for a drone delivering a package from a moving truck. Additionally, we propose a wind-biased RRT planner that nudges sampled points along the wind direction to find more efficient routes, and pair it with a QP-based linear MPC controller to handle real-time wind disturbances during flight. We compare our approach against standard RRT and A* across five map configurations, evaluating airtime, energy consumption, and planning time.**

Project Code: https://github.com/vivekdhir77/Custom-RRT-and-Model-Predictive-Control

***Index Terms*—Path Planning, Model Predictive Control, Joint Optimization**

I. Introduction

The Rapid growth of e-commerce has made last-mile delivery one of the most difficult and expensive part of shipping process. In many cases this final stage of shipping process takes large amount total delivery time and cost, mainly in crowded areas where trucks can be slowed down by traffic and multiple delivery locations. Because of this Truck-Drone delivery become as attractive idea. In such system, Truck with drone moves around the city, Drone takes off from truck to deliver package to customer and than return back to Truck which can be stationary or moving. This setup combines strength of both vehicles, truck can carry all the packages while drone can deliver those packages to customer avoiding traffic congestion.

Even though this idea is promising, using it in real life is not straight forward. Drone cannot fly in ideal condition. Factor like wind can affect speed, flight time, energy use and ability to return to truck accurately. The problem becomes even more challenging when the truck is moving during drone's flight time. In that case Drone has to choose a good launch time for delivery and reliable point to rejoin the truck. As a result simply finding shortest route is not enough. The system also needs a path planner that considers the wind and a controller that can track the path safely and accurately.

Existing work on collaborative truck-drone delivery has mainly focused on route optimization. Murray and Chu [1] introduced this idea as the Flying Sidekick Traveling Salesman Problem (FSTSP), and later studies extended it to multi-drone systems [2] and moving launch and recovery settings [3], [4]. However, most of these studies assume that the drone travels along a simple straight-line path and do not consider how environmental effects such as wind can change the drone's motion.

On the path planning side, graph-based methods such as A star [12] can find geometrically short paths, but they do not account for wind and usually do not focus on smooth or efficient flight. Variants such as Theta star [13] and adaptive heuristic methods [14] have similar limitations. Sampling-based methods are more flexible. For example, RRT+ provides asymptotic optimality [5], Informed RRT+ improves convergence by sampling more effectively [15], and Kinodynamic RRT+ includes system dynamics to generate feasible quadrotor trajectories [16]. Some studies have introduced wind awareness into planning by changing A star cost functions [7] or using physics-informed neural networks [8]. However, to the best of our knowledge, previous work has not directly biased the RRT sampling process using the drone's expected ground-frame motion under wind.

For trajectory tracking, Model Predictive Control (MPC) is widely used for quadrotors because it can handle system constraints in a systematic way [9]. In windy conditions similar to truck-drone rendezvous, Paris et al. [10] showed that MPC can be used for landing on a moving platform. However, that work did not address upstream path planning or launch-time optimization.

In this paper, we study the combination problem of planning and control of drone that takes of from moving truck at best time, delivers the package and then returns and rejoins the truck at planned meet point. Our Approach has 3 main parts: (1) Searching for the best launch time, (2) Using a wind biased RRT planner to find a better route in windy conditions, and (3) Using a QP-based linear MPC controller to help the drone follow the planned path while staying within the speed and acceleration limits. Wind is modeled as an Ornstein-Uhlenbeck process, and only its average value is used during planning. Random wind changes are not directly included in the planner, and are only handled through MPC feedback during flight. We compare the proposed wind-biased RRT with standard RRT and A* using the same MPC tracking setup, and we evaluate them based on airtime, energy use, path length, and computation time.

II. Problem Formulation

A. System Overview

A drone stationed on a moving truck must deliver a package to a fixed off-road delivery point $\mathbf{x}_p$ and return to land back on the same truck, which never stops moving. The operating environment is modelled as a 1000×1000 cell grid, where each cell covers $1\,\mathrm{m} \times 1\,\mathrm{m}$ of physical space. Because the truck and drone are subject to different types of restricted zones, separate occupancy grids are needed for each agent. Throughout the flight, a persistent wind field modelled as an Ornstein-Uhlenbeck (OU) process acts on the drone, changing its effective ground speed depending on the direction of travel.

Three decisions must be made jointly: *when* to launch the drone, *which path* to fly given the wind conditions, and *how* to track that path in the presence of stochastic wind disturbances. All symbols used in this section are listed in Table I.

TABLE I
NOTATION

Symbol	Space	Description
k	$\mathbb{N}_0$	Discrete time slot index
T	$\mathbb{R}_+$	Time step duration (0.5 s)
$t_k = Tk$	$\mathbb{R}_+$	Continuous time at slot k
k_L^*	$\mathbb{N}_0$	Optimal drone launch step
k_d	$\mathbb{N}_0$	Delivery step
k_R	$\mathbb{N}_0$	Landing step
$\gamma(s)$	$\mathbb{R}^2$	Road position at arc-length s
s_k	$\mathbb{R}$	Truck arc-length at step k
v_T	$\mathbb{R}_+$	Truck constant speed
V_{cruise}	$\mathbb{R}_+$	Drone cruise speed
v_{eff}	$\mathbb{R}_+$	Wind-adjusted drone speed
$\mathbf{p}_{T,k}$	$\mathbb{R}^2$	Truck position at step k
$\mathbf{p}_{d,k}$	$\mathbb{R}^2$	Drone position at step k
$\mathbf{u}_{d,k}$	$\mathbb{R}^2$	Drone control acceleration
m_0	$\mathbb{R}_+$	Initial drone mass (1.0 kg)
$m(k)$	$\mathbb{R}_+$	Drone mass at step k
α	$(0,1)$	Mass ratio after package drop (0.75)
$\mathbf{w}_k$	$\mathbb{R}^2$	Wind velocity at step k
α_w	$\mathbb{R}_+$	OU wind mean-reversion rate (0.4)
$\bar{\mathbf{w}}$	$\mathbb{R}^2$	OU wind mean vector
σ_{OU}	$\mathbb{R}_+$	OU wind noise std (0.3)
$\boldsymbol{\xi}_k$	$\mathbb{R}^2$	Standard Gaussian noise at step k
$\mathbf{x}_p$	$\mathbb{R}^2$	Delivery point location
$\mathcal{G}_T$	$\{0,1\}^{N\times N}$	Truck occupancy grid
$\mathcal{G}_D$	$\{0,1\}^{N\times N}$	Drone occupancy grid
m_T, m_D	$\mathbb{N}$	Safety margins (2, 14 cells)
T_{out}	$\mathbb{R}_+$	Outbound flight time
T_{ret}	$\mathbb{R}_+$	Return flight time
$J(k_L)$	$\mathbb{R}_+$	Total path length objective
L_{out}, L_{ret}	$\mathbb{R}_+$	Outbound and return path lengths
Δ_s	$\mathbb{N}$	Launch search coarse step (10)
λ	$\mathbb{R}_+$	Return time multiplier
$K_{\max}$	$\mathbb{N}$	Maximum search horizon
C_{truck}	$\mathbb{R}_+$	Truck path planning cost
C_{drone}	$\mathbb{R}_+$	Drone path planning cost per call
ε_D	$\mathbb{R}_+$	Delivery position tolerance (10 m)
$\varepsilon_{\text{land}}$	$\mathbb{R}_+$	Landing position tolerance (15 m)
$E(k)$	$\mathbb{R}_+$	Cumulative mechanical energy

B. Problem Statement

Time is divided into discrete slots indexed by $k \in \mathbb{N}_0$, with each slot lasting $T = 0.5$ s, so the continuous time at slot k is $t_k = Tk$. The drone departs the truck at the optimal launch step k_L^*, drops the package at step k_d, and lands back on the truck at step k_R. The goal is to find k_L^* that minimises the total planned path length:

$$k_L^* = \arg\min_{k_L} J(k_L), \qquad J(k_L) = L_{\text{out}}(k_L) + L_{\text{ret}}(k_L) \tag{1}$$

This is subject to three constraints: the drone's outbound and return paths must be collision-free on $\mathcal{G}_D$; the drone must successfully intercept the moving truck on return; and the drone cruise speed must satisfy $V_{\text{cruise}} > v_T$, which is proved in Section II-C.

C. Relevance and Justification

1) Why Two Separate Occupancy Grids: The truck is bound to a road network and must avoid obstacles such as buildings and pedestrian zones. The drone, on the other hand, flies in open airspace and must respect no-fly zones. These two sets of constraints do not overlap. Using a single shared grid would either block road segments the truck legitimately needs, or leave the drone exposed to hazardous regions. Separate grids $\mathcal{G}_T$ and $\mathcal{G}_D$ with margins $m_T = 2$ and $m_D = 14$ cells let each agent plan within its own feasible space. The drone margin is set larger because the drone requires more clearance when manoeuvring in open airspace compared to the truck following a constrained road.

2) Why Launch Timing is Critical: If the drone launches too early, it travels a long outbound path and by the time it finishes delivery, the truck is far ahead, making the return leg difficult. If it launches too late, the truck is close to its destination and there is not enough road left for a successful intercept. The optimal k_L^* sits between these extremes, keeping both L_{out} and L_{ret} short enough to minimise $J(k_L)$.

3) Why Wind Matters: Wind affects the drone's ground speed in a direction-dependent way. A tailwind shortens flight time; a headwind lengthens it. Two geometrically identical paths can therefore take different amounts of time depending on how they are oriented relative to the mean wind vector $\bar{\mathbf{w}}$. Planning purely for shortest geometric distance, as A* does, can produce routes that fight the wind unnecessarily. This is the motivation for comparing A* against wind-aware planners under the same MPC tracking setup.

4) Why A for the Truck Path, Not RRT:* The truck moves along a fixed road network, which is already encoded as the grid graph $\mathcal{G}_T$. A* finds the shortest path on this graph in $O(N^2 \log N)$ time and always returns the same result. RRT, by contrast, builds its tree by sampling random points across the full 2D space, with runtime $O(N_{\text{iter}} \cdot |\mathcal{P}|)$ that grows with the number of iterations N_{iter} and the size of the nearest-neighbour search space $|\mathcal{P}|$. The launch search must evaluate $\lfloor K_{\max}/\Delta_s \rfloor$ candidate steps, calling the drone planner at each one, so the total planning cost is:

$$C_{\text{total}} = C_{\text{truck}} + \frac{K_{\max}}{\Delta_s} \cdot C_{\text{drone}} \tag{2}$$

where C_{truck} is the truck path planning cost and C_{drone} is the cost of one drone planner call. Using A* keeps C_{truck}

small and fixed, computed only once. Applying RRT to the truck path would waste computation sampling 2D space for a problem that is already a 1D graph traversal. The drone, flying freely in 2D under wind, is where the flexibility of RRT is actually useful.

5) Drone Must Fly Faster Than the Truck: During the drone's round trip, the truck covers some distance d_{truck} along the road while the drone covers outbound distance d_{out} and return distance d_{ret}. Since both must complete their journeys in the same time window:

$$\frac{d_{\text{out}} + d_{\text{ret}}}{V_{\text{cruise}}} = \frac{d_{\text{truck}}}{v_T} \quad \Longrightarrow \quad V_{\text{cruise}} = v_T \cdot \frac{d_{\text{out}} + d_{\text{ret}}}{d_{\text{truck}}} \tag{3}$$

The drone travels two legs while the truck travels one. Since two sides of any path are always strictly longer than the remaining single segment:

$$d_{\text{out}} + d_{\text{ret}} > d_{\text{truck}} \quad \Longrightarrow \quad \boxed{V_{\text{cruise}} > v_T} \tag{4}$$

The further the delivery point lies from the truck road, the larger the required speed difference becomes.

D. Objectives

Finding k_L^* requires three things to be computable: the truck's position at any future time, the wind-adjusted travel speed along any path segment, and the path length estimates for both legs.

1) Truck Path Parameterisation: The truck path is a sequence of waypoints $\{\mathbf{w}_i\}_{i=0}^{N_w}$, computed by A* on $\mathcal{G}_T$. The cumulative arc-length s_i records the total path distance from the start up to waypoint $\mathbf{w}_i$. Given any arc-length value $s \in [s_i, s_{i+1}]$, the physical position on the path is found by linearly interpolating between the two surrounding waypoints:

$$\gamma(s) = \mathbf{w}_i + \frac{s - s_i}{s_{i+1} - s_i}(\mathbf{w}_{i+1} - \mathbf{w}_i) \tag{5}$$

Since the truck moves at constant speed v_T, the arc-length covered by step k is $v_T \cdot t_k$, so the truck position at any step is simply:

$$\mathbf{p}_{T,k} = \gamma(v_T \cdot t_k) \tag{6}$$

This lets the launch search query the truck's future position at any candidate time without replanning the path.

2) Wind-Adjusted Effective Speed: For a drone flying from point $\mathbf{a}$ toward point $\mathbf{b}$, the wind component along the direction of travel is projected out and added to the cruise speed:

$$v_{\text{eff}}(\mathbf{a}, \mathbf{b}) = \max\left(V_{\text{cruise}} + \bar{\mathbf{w}} \cdot \frac{\mathbf{b} - \mathbf{a}}{\|\mathbf{b} - \mathbf{a}\|},\ 0\right) \tag{7}$$

The $\max(\cdot, 0)$ prevents the effective speed from going negative under a strong headwind. This is the reason two paths of equal geometric length can have different flight times — it depends entirely on how much of each segment aligns with or against the wind.

3) Outbound Path Length and Return Intercept: For a candidate launch step k_L, the drone departs from position $\mathbf{p}_L = \gamma(v_T \cdot t_{k_L})$, which is the truck's location at that moment. The outbound flight time to $\mathbf{x}_p$ over a path of length $L_{\text{out}}(k_L)$ is:

$$T_{\text{out}}(k_L) = \frac{L_{\text{out}}(k_L)}{v_{\text{eff}}(\mathbf{p}_L,\, \mathbf{x}_p)} \tag{8}$$

By the time the drone finishes its outbound leg, the truck has moved further along the road. The truck arc-length at that moment is $s_{\text{del}} = s_{k_L} + T_{\text{out}} \cdot v_T$, where $s_{k_L} = v_T \cdot t_{k_L}$ is the truck arc-length at launch. Since the truck keeps moving during the return leg as well, the rendezvous point cannot be computed exactly without knowing the return flight time, which itself depends on the rendezvous point. To break this circular dependency, the rendezvous arc-length is estimated by sweeping a return time multiplier $\lambda \in \{1.0, 1.5, 2.0, 2.5, 3.0\}$:

$$s_{\text{rv}}(\lambda) = s_{k_L} + (1 + \lambda) \cdot T_{\text{out}} \cdot v_T \tag{9}$$

For each value of λ, the rendezvous position $\mathbf{p}_{\text{rv}} = \gamma(s_{\text{rv}}(\lambda))$ is recovered from the arc-length map, and a return path of length $L_{\text{ret}}(\lambda)$ is planned from $\mathbf{x}_p$ to $\mathbf{p}_{\text{rv}}$ on $\mathcal{G}_D$.

4) Two-Phase Launch Search: With $J(k_L, \lambda) = L_{\text{out}}(k_L) + L_{\text{ret}}(k_L, \lambda)$ defined for any combination of launch step and return multiplier, the optimal launch step is:

$$k_L^* = \arg \min_{\substack{k_L \in \{0,\, \Delta_s,\, \ldots,\, K_{\max}\} \\ \lambda \in \{1.0,\, 1.5,\, 2.0,\, 2.5,\, 3.0\}}} J(k_L, \lambda) \tag{10}$$

Evaluating every step individually would be expensive. The coarse step $\Delta_s = 10$ reduces the number of planner calls to roughly $K_{\max}/\Delta_s \times |\Lambda|$, where $|\Lambda| = 5$ is the number of multiplier candidates, while still finding the optimal launch step to within Δ_s precision.

E. Requirements and Constraints

1) Flight Phase Definitions: The mission is divided into four phases. During **prelaunch** ($k < k_L^*$), the drone sits on the truck and no control input is applied. The **outbound** phase ($k_L^* \leq k < k_d$) begins at launch and ends when the drone gets close enough to the delivery point:

$$k_d = \min\{k \geq k_L^* : \|\mathbf{p}_{d,k} - \mathbf{x}_p\| < \varepsilon_D\} \tag{11}$$

The **return** phase ($k_d \leq k \leq k_R$) begins after delivery and ends when the drone is close enough to the truck to land:

$$k_R = \min\{k \geq k_d : \|\mathbf{p}_{d,k} - \mathbf{p}_{T,k}\| < \varepsilon_{\text{land}}\} \tag{12}$$

After landing, the **postlanding** phase ($k > k_R$) begins — the drone rides on the truck until the truck reaches its final destination. The total drone airtime across the mission is:

$$\Delta t = (k_R - k_L^*) \cdot T \tag{13}$$

2) Wind Model: Throughout all phases, the wind state evolves according to a discrete OU process:

$$\mathbf{w}_{k+1} = (1 - \alpha_w T)\mathbf{w}_k + \alpha_w T \bar{\mathbf{w}} + \sigma_{\mathrm{OU}} \sqrt{T}\, \boldsymbol{\xi}_k \tag{14}$$

where $\boldsymbol{\xi}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is independent Gaussian noise, $\alpha_w = 0.4$ controls how quickly the wind reverts to its mean $\bar{\mathbf{w}}$, and $\sigma_{\mathrm{OU}} = 0.3$ sets the magnitude of random gusts. Path planning treats $\bar{\mathbf{w}}$ as a fixed deterministic input; the stochastic fluctuations are left for the MPC to handle at runtime through its feedback mechanism.

3) Package Drop and Mass Variation: When the drone reaches $\mathbf{x}_p$ and delivers the package at step k_d, its mass drops immediately:

$$m(k) = \begin{cases} m_0 & k < k_d \\ \alpha \cdot m_0 & k \geq k_d \end{cases} \tag{15}$$

where $m_0 = 1.0$ kg is the loaded mass and $\alpha = 0.75$ is the ratio of drone mass to total loaded mass. After the drop, the same thrust command produces more acceleration, which is reflected directly in the energy profile.

F. Evaluation Criteria

All three planners: A*, standard RRT, and wind-biased RRT, are run under the same MPC tracking setup and compared on four metrics:

Total Airtime: the time from launch to landing, $\Delta t = (k_R - k_L^*) \cdot T$ seconds.

Mechanical Energy: the cumulative propulsive work done by the drone over the full flight:

$$E(k) = \sum_{i=0}^{k-1} \max\big(m(i)\, \mathbf{u}_{d,i}^{\top} \mathbf{v}_{d,i},\ 0\big) \cdot T \tag{16}$$

where $m(i)$ follows (15), $\mathbf{u}_{d,i} \in \mathbb{R}^2$ is the drone control acceleration, and $\mathbf{v}_{d,i} \in \mathbb{R}^2$ is the drone velocity at step i. Only positive power is counted; the mass drop at k_d reduces energy consumption for the same thrust on the return leg.

Path Length: total planned distance $L_{\mathrm{out}} + L_{\mathrm{ret}}$ in metres.

Planning Time: wall-clock time in milliseconds to compute the outbound and return paths for each planner.

III. Methodology

This section describes the implementation pipeline that realises the formulation of Section II. The pipeline has three sequential stages, each corresponding to a software module executed in order: environment construction, path planning with launch optimisation, and MPC-based trajectory tracking. Fig. 1 provides a schematic overview.

A. Stage 1: Environment Construction

The two occupancy grids $\mathcal{G}_T$ and $\mathcal{G}_D$ are built from the obstacle list by marking every cell within the safety margin of any obstacle as blocked, using margins $m_T = 2$ and $m_D = 14$ cells respectively, as motivated in Section II-C. As visible in Fig. 1, the margin inflation produces the red offset regions surrounding each restricted zone. The three key points:

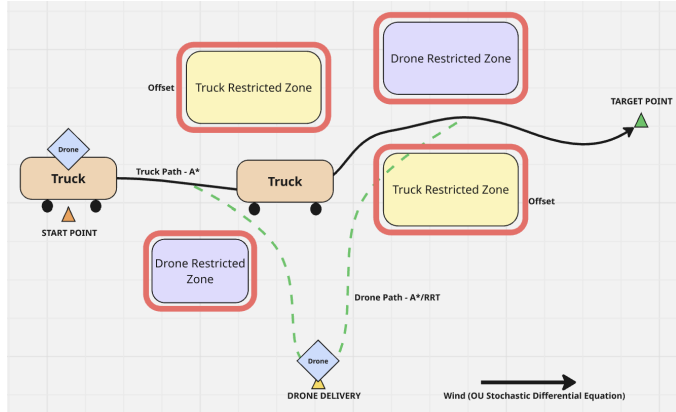

Fig. 1. System overview of the truck-drone collaborative delivery. The truck follows an A*-planned road path (solid line) while the drone launches at the optimal time, flies to the delivery point via A*, RRT or wind-biased RRT (dashed line), and returns to rendezvous with the moving truck. Shaded regions denote restricted zones for each agent, with red borders indicating the safety margin (offset) used during occupancy grid construction. The wind arrow denotes the prevailing wind modeled as an Ornstein-Uhlenbeck process.

truck start $\mathbf{p}_s^{\mathrm{truck}}$, truck end $\mathbf{p}_e^{\mathrm{truck}}$, and drone delivery point $\mathbf{x}_p$ are validated to ensure they fall in passable cells on their respective grids. The grids and configuration are serialised to disk for use by the downstream stages.

B. Stage 2: Path Planning and Launch Optimisation

This stage computes the truck path, determines the optimal drone launch index, and plans the drone's outbound and return paths.

1) Truck Path: The truck's shortest path on $\mathcal{G}_T$ is computed once via A*, producing a waypoint sequence $\mathcal{P}_{\mathrm{truck}}$. The cumulative arc-length and position map in (5) are constructed over this path to allow truck position queries at any future time via (6).

2) Launch Search and Rendezvous Computation: The two-phase search (10) is executed: for each candidate launch step $k_L \in \{0, \Delta_s, 2\Delta_s, \ldots, K_{\max}\}$ and each return multiplier $\lambda \in \{1.0, 1.5, 2.0, 2.5, 3.0\}$, an A* outbound path is planned on $\mathcal{G}_D$ from $\mathbf{p}_{T,k_L}$ to $\mathbf{x}_p$. The outbound flight time is estimated as $T_{\mathrm{out}} = L_{\mathrm{out}}/V_{\mathrm{cruise}}$, and the rendezvous cell $\mathbf{p}_{\mathrm{rv}}$ is recovered from (9). A return path of length L_{ret} is then planned from $\mathbf{x}_p$ to $\mathbf{p}_{\mathrm{rv}}$ on $\mathcal{G}_D$. The pair (k_L^*, λ^*) minimising $J(k_L, \lambda) = L_{\mathrm{out}} + L_{\mathrm{ret}}$ from (10) is selected. The optimal launch index k_L^*, the corresponding rendezvous cell $\mathbf{p}_{\mathrm{rv}}^*$, and the A* outbound/return paths are stored for use by all three drone planners.

3) Drone Path Planning: Three planners are run on the outbound and return legs using the launch point $\mathbf{p}_{T,k_L^*}$ and rendezvous point $\mathbf{p}_{\mathrm{rv}}^*$ from the search above:

- **A*** is a standard grid-based shortest-path search on $\mathcal{G}_D$ using 8-connected neighbours and the Euclidean heuristic. Produces the geometrically shortest collision-free path but is wind-agnostic.
- **Standard RRT** is a baseline sampling-based planner that grows a tree from the start toward uniformly random samples on $\mathcal{G}_D$, extending the nearest node by a fixed

step size δ until the goal region is reached. No wind awareness.

- **Wind-biased RRT** is a sample-shifting variant of RRT. Let $\hat{w} = \bar{\mathbf{w}}/\|\bar{\mathbf{w}}\|$ be the mean wind direction and let δ_w be a fixed shift magnitude in cells. With probability 0.1, a drawn random sample $\mathbf{q}_{\text{rand}}$ is replaced by its shifted version

$$\mathbf{q}'_{\text{rand}} = \mathbf{q}_{\text{rand}} + \delta_w \, \hat{w}_{\text{eff}}, \tag{17}$$

where $\hat{w}_{\text{eff}} = \hat{w}$ on the outbound leg and $\hat{w}_{\text{eff}} = -\hat{w}$ on the return leg. Negating the shift on the return leg biases the planned path upwind so that actual wind drift during execution corrects the trajectory toward the rendezvous point.

All three produce a collision-free waypoint sequence that is passed to the MPC stage. The same launch index k_L^* and rendezvous point $\mathbf{p}_{\text{rv}}^*$ are used for all planners to ensure a fair comparison.

C. Stage 3: MPC Trajectory Tracking

The 10-state joint MPC formulation of Section III-D tracks each planned drone path while simultaneously modelling the truck and wind dynamics.

1) Reference Trajectory Generation: Each planned waypoint sequence (from A*, standard RRT, or wind-biased RRT) is converted to a time-parameterised reference trajectory. The waypoint cells are scaled to metres, and a cumulative arc-length s is computed over the path. The total number of reference steps is $N_{\text{ref}} = \lceil s_{\text{total}}/(V_{\text{cruise}} \cdot T) \rceil + 1$. At each step k, the reference position is obtained by linear interpolation along the path at distance $s_k = k \cdot V_{\text{cruise}} \cdot T$, and the reference velocity is set as the path tangent scaled by V_{cruise}. The truck reference is generated identically using v_T along $\mathcal{P}_{\text{truck}}$. The wind reference is set to the OU steady-state mean $\bar{\mathbf{w}}$ at every horizon step.

2) Simulation Loop: The simulation proceeds through the four flight phases defined in Section II-E:

a) Pre-launch ($k < k_L^*$): The drone rides on the truck with no control input. The wind state evolves stochastically via (14), establishing realistic initial wind conditions before flight begins.

b) Outbound ($k_L^* \le k < k_d$): The MPC solves the QP at each step using the outbound reference trajectory. The first two elements of $\mathbf{U}^*$ are applied as drone acceleration; the truck acceleration commands are also computed but the truck position is overwritten with ground-truth kinematic propagation along $\mathcal{P}_{\text{truck}}$ at constant speed v_T. The stochastic OU noise term $\sigma_{\text{OU}}\sqrt{T}\,\boldsymbol{\xi}_k$ is added to the wind state at every propagation step, so the MPC must reject real-time wind disturbances that were not present in the planned path. Delivery is declared when the proximity condition (11) is met, and the drone mass drops to $\alpha \cdot m_0$ per (15).

c) Return ($k_d \le k \le k_R$): The MPC switches to the return reference trajectory. The truck position continues to be updated kinematically. Landing is declared when (12) is satisfied.

d) Post-landing ($k > k_R$): The drone is co-located with the truck. The simulation continues until the truck reaches $\mathbf{p}_e^{\text{truck}}$.

3) Logged Outputs: At every step the simulation records the full 10-state vector, drone control inputs, current phase, and instantaneous mass. After completion, the cumulative energy (16), total airtime (13), and per-step QP solve times are computed. These quantities form the evaluation metrics defined in Section II-F and are reported for all three planners under identical conditions in Section V.

D. Model Predictive Control Formulation

For the local planner, the drone delivery system is controlled using a Model Predictive Controller (MPC) that jointly optimizes the drone and truck trajectories while tracking precomputed reference paths (A* or RRT). The controller runs at a sampling time of $T = 0.5$ s with prediction horizon $F = 20$ and control horizon $V_{\text{HOR}} = 20$.

1) Continuous-Time Models: **Drone dynamics (double integrator with wind bias):** The drone state is $\mathbf{x}_D = [p_{D_x},\ p_{D_y},\ v_{D_x},\ v_{D_y},\ w_x,\ w_y]^\top$. Where the position of the drone is given by $(p_{D_x},\ p_{D_y})$, velocity of the drone is given by $(v_{D_x},\ v_{D_y})$, wind induced acceleration is given by $(w_{D_x},\ w_{D_y})$. Its continuous time equations are

$$\dot{\mathbf{p}}_D = \mathbf{v}_D, \tag{18}$$

$$\dot{\mathbf{v}}_D = \mathbf{u}_d + \mathbf{w}, \tag{19}$$

$$\dot{\mathbf{w}} = -a_w(\mathbf{w} - \bar{\mathbf{w}}), \tag{20}$$

where $\mathbf{u}_d \in \mathbb{R}^2$ is the control acceleration (mass $m = 1$), $\mathbf{w}$ is the wind induced acceleration, and $a_w = 0.4$ (mean reversion rate), $\bar{\mathbf{w}} = [0.8,\ 0.3]^\top$.

Truck dynamics: The truck state is given by $\mathbf{x}_T = [p_{T_x},\ p_{T_y},\ v_{T_x},\ v_{T_y}]^\top$. Its continuous-time equations are

$$\dot{\mathbf{p}}_T = \mathbf{v}_T, \tag{21}$$

$$\dot{\mathbf{v}}_T = \mathbf{a}_T, \tag{22}$$

where $\mathbf{a}_T \in \mathbb{R}^2$ is the truck acceleration command.

2) Derivation of the Discrete-Time System Model: The combined state is

$$\mathbf{x}(k) = \begin{bmatrix}\mathbf{x}_D(k)^\top & \mathbf{x}_T(k)^\top\end{bmatrix}^\top \in \mathbb{R}^{10}.$$

The control input is

$$\mathbf{u}(k) = \begin{bmatrix}\mathbf{u}_d(k)^\top & \mathbf{a}_T(k)^\top\end{bmatrix}^\top \in \mathbb{R}^4.$$

Drone discretization: A constant control is given at a time interval (also called zero order hold) is given by integrating (18)–(19) exactly on $\mathbf{u}_d$, which gives the double integrator update:

$$\mathbf{p}_D(k+1) = \mathbf{p}_D(k) + T\,\mathbf{v}_D(k) + \frac{T^2}{2}\,\mathbf{u}_d(k), \tag{23}$$

$$\mathbf{v}_D(k+1) = \mathbf{v}_D(k) + T\,\mathbf{u}_d(k) + T\,\mathbf{w}(k). \tag{24}$$

Applying forward Euler to the wind dynamics (20) yields

$$\mathbf{w}(k+1) = (1 - a_w T)\,\mathbf{w}(k) + a_w T\,\bar{\mathbf{w}}.$$

Collecting all six drone states produces the block

$$A_D = \begin{bmatrix} 1 & 0 & T & 0 & T^2/2 & 0 \\ 0 & 1 & 0 & T & 0 & T^2/2 \\ 0 & 0 & 1 & 0 & T & 0 \\ 0 & 0 & 0 & 1 & 0 & T \\ 0 & 0 & 0 & 0 & 1-a_wT & 0 \\ 0 & 0 & 0 & 0 & 0 & 1-a_wT \end{bmatrix}$$

$$B_D = \begin{bmatrix} T^2/2 & 0 \\ 0 & T^2/2 \\ T & 0 \\ 0 & T \\ 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

The constant wind drive term for the drone is $\mathbf{d}_D = [0,\ 0,\ 0,\ 0,\ a_wT\bar{w}_x,\ a_wT\bar{w}_y]^\top$.

Truck discretization : A ZOH of the model is given by discretization of the wind and integration of the double order equation i.e. Exact integration of (21)–(22) gives

$$A_T = \begin{bmatrix} 1 & 0 & T & 0 \\ 0 & 1 & 0 & T \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad B_T = \begin{bmatrix} T^2/2 & 0 \\ 0 & T^2/2 \\ T & 0 \\ 0 & T \end{bmatrix}, \quad \mathbf{d}_T = \mathbf{0}.$$

Combined discrete-time model: Assembling the blocks yields the 10-state linear system

$$\mathbf{x}(k+1) = A\mathbf{x}(k) + B\mathbf{u}(k) + \mathbf{d}, \tag{25}$$

with block-diagonal $A = \mathrm{blkdiag}(A_D, A_T)$ and input matrix

$$B = \begin{bmatrix} B_D & \mathbf{0}_{6\times 2} \\ \mathbf{0}_{4\times 2} & B_T \end{bmatrix}$$

and constant disturbance '$\mathbf{d}$'. The wind bias relaxes toward $\bar{\mathbf{w}}$ with discrete pole $1 - a_wT = 0.8$, corresponding to a continuous-time constant $\tau = 1/a_w = 2.5$ s.

3) Prediction Model: Given the current state $\mathbf{x}_0 = \mathbf{x}(k)$, the stacked predicted states over the horizon F are $\mathbf{X} = [\mathbf{x}_{k+1|k}^\top \ \cdots \ \mathbf{x}_{k+F|k}^\top]^\top$. Using (25) the prediction is

$$\mathbf{X} = \mathcal{O}\mathbf{x}_0 + \mathcal{M}_m\mathbf{U} + \mathcal{D}\mathbf{d},$$

where $\mathbf{U} = [\mathbf{u}_k^\top \ \cdots \ \mathbf{u}_{k+V_{\mathrm{HOR}}-1}^\top]^\top \in \mathbb{R}^{V_{\mathrm{HOR}}\cdot 4}$ is the decision variable, and $\mathcal{O}$, $\mathcal{M}_m$, $\mathcal{D}$ are the (precomputed) lifted prediction matrices obtained by stacking powers of A, the convolution terms with B, and cumulative disturbance effects, respectively.

The reference trajectory $\mathbf{Y}_d \in \mathbb{R}^{F\cdot 10}$ is generated by time-parameterizing the precomputed drone outbound/return paths and the truck path (constant speed $V_T = 5$ m/s). In the return phase, the final 20 % of the drone reference path is linearly blended toward the live predicted truck position to enable smooth rendezvous. Lets define the resultant trajectory without controls as the following:

$$\mathbf{S}_k = \mathbf{Y}_d - \mathcal{O}\mathbf{x}_0 - \mathcal{D}\mathbf{d}.$$

The tracking error then simplifies to

$$\mathbf{X} - \mathbf{Y}_d = \mathcal{M}_m\mathbf{U} - \mathbf{S}_k.$$

4) Quadratic Cost Function and Quadratic Program: The finite-horizon cost is

$$J(\mathbf{U}) = (\mathbf{X} - \mathbf{Y}_d)^\top W_4(\mathbf{X} - \mathbf{Y}_d) + \mathbf{U}^\top W_3\mathbf{U},$$

where W_4 is block-diagonal with stage cost

$$Q = \mathrm{diag}(500, 500, 5, 5, 0, 0, 30, 30, 1, 1)$$

for the first $F-1$ blocks and the terminal cost Q_f (solution of the discrete algebraic Riccati equation) on the final block. The control cost matrix W_3 is block-diagonal with

$$R = \mathrm{diag}(0.1, 0.1, 0.3, 0.3).$$

Substituting the prediction model yields the standard quadratic program

$$J(\mathbf{U}) = \tfrac{1}{2}\mathbf{U}^\top H\mathbf{U} + \mathbf{f}_k^\top\mathbf{U} + \text{const},$$

with

$$H = 2(\mathcal{M}_m^\top W_4\mathcal{M}_m + W_3), \qquad \mathbf{f}_k = -2\mathcal{M}_m^\top W_4\mathbf{S}_k.$$

All matrices H, G, etc., are precomputed offline.

5) Unconstrained LQ Solution and Constrained QP: Ignoring constraints, the optimal control sequence is

$$\mathbf{U}^* = G\mathbf{S}_k, \quad G = (\mathcal{M}_m^\top W_4\mathcal{M}_m + W_3)^{-1}\mathcal{M}_m^\top W_4.$$

6) Box Constraints: To ensure real life constrains for feasibility and safe operation, we incorporate box constrains on the control inputs and predicted velocity states over the entire horizon $V_{\mathrm{HOR}} = 20$.

a) Control input constraints: The decision variable is the stacked control sequence

$$\mathbf{U} = \left[\mathbf{u}_k^\top,\ \mathbf{u}_{k+1}^\top,\ \ldots,\ \mathbf{u}_{k+V_{\mathrm{HOR}}-1}^\top\right]^\top \in \mathbb{R}^{V_{\mathrm{HOR}}\cdot M}, \quad M = 4,$$

where each $\mathbf{u}_{k+i} = [u_{d_x},\ u_{d_y},\ a_{T_x},\ a_{T_y}]^\top$ contains the drone $(u_{d_x},\ u_{d_y})$ and truck $(a_{T_x},\ a_{T_y})$ acceleration inputs. The following bounds are imposed on every element of $\mathbf{U}$:

$$u_{d_x},\ u_{d_y} \in [U_{D_{\min}},\ U_{D_{\max}}],$$
$$a_{T_x},\ a_{T_y} \in [A_{T_{\min}},\ A_{T_{\max}}].$$

These bounds are represented as the following:

$$\mathbf{U}_{\min} = \mathrm{tile}\big([U_{D_{\min}},\ U_{D_{\min}},\ A_{T_{\min}},\ A_{T_{\min}}],\ V_{\mathrm{HOR}}\big), \tag{26}$$
$$\mathbf{U}_{\max} = \mathrm{tile}\big([U_{D_{\max}},\ U_{D_{\max}},\ A_{T_{\max}},\ A_{T_{\max}}],\ V_{\mathrm{HOR}}\big), \tag{27}$$

through which we enforce the box constraint

$\mathbf{U}_{\min} \le \mathbf{U} \le \mathbf{U}_{\max}$.

b) Velocity state constraints: The predicted state sequence over the horizon satisfies

$$\mathbf{X} = \mathcal{O}\,\mathbf{x}_0 + \mathcal{M}_m\,\mathbf{U} + \mathcal{D}\,\mathbf{d} \ \in\ \mathbb{R}^{F\cdot N},$$

where $\mathcal{O}$, $\mathcal{M}_m$, and $\mathcal{D}$ are the observability, control-to-output, and disturbance matrices, respectively.

The following defines velocity constrains. These are enforced on every predicted state $\mathbf{x}_{k+i|k}$, $i = 1, \ldots, F$, at the relevant state:

$$\begin{aligned} v_{D_x},\ v_{D_y} &\in [V_{D_{\min}},\ V_{D_{\min}}], \\ v_{T_x} &\in [V_{T_{X_{\min}}},\ V_{T_{X_{\max}}}], \\ v_{T_y} &\in [V_{T_{Y_{\min}}},\ V_{T_{Y_{\max}}}]. \end{aligned}$$

Position and wind states are left unconstrained. Defining the bound vectors $\mathbf{X}_{\min}, \mathbf{X}_{\max} \in \mathbb{R}^{F \cdot N}$ with the above limits at the velocity indices and $\pm 10^6$ at all other positions. The state constraint is defined like the following:

$$\mathbf{X}_{\min} \ \leq\ \mathcal{O}\,\mathbf{x}_0 + \mathcal{M}_m\,\mathbf{U} + \mathcal{D}\,\mathbf{d} \ \leq\ \mathbf{X}_{\max}.$$

c) Transformation to standard inequality form: To express the state constraints as linear inequalities in $\mathbf{U}$, wedefine the free response like the following:

$$\mathbf{Y}_{\text{bar}} \triangleq \mathcal{O}\,\mathbf{x}_0 + \mathcal{D}\,\mathbf{d},$$

i.e. the predicted state trajectory under zero control input. Subtracting $\mathbf{Y}_{\text{bar}}$ from all sides gives

$$\mathbf{X}_{\min} - \mathbf{Y}_{\text{bar}} \ \leq\ \mathcal{M}_m\,\mathbf{U} \ \leq\ \mathbf{X}_{\max} - \mathbf{Y}_{\text{bar}}.$$

Multiplying the lower bound through by -1 and reversing the inequality converts both sides to standard upper-bound form, yielding the matrix inequality

$$\begin{bmatrix} -\mathcal{M}_m \\ \mathcal{M}_m \end{bmatrix} \mathbf{U} \ \leq\ \begin{bmatrix} \mathbf{Y}_{\text{bar}} - \mathbf{X}_{\min} \\ \mathbf{X}_{\max} - \mathbf{Y}_{\text{bar}} \end{bmatrix} \ \triangleq\ \mathbf{b}_x(k).$$

The right-hand side vector $\mathbf{b}_x(k)$ is state-dependent and is recomputed at every time step k.

d) Full constraint matrix.: Combining the input and state constraints, the complete set of linear inequalities passed to the quadratic polynomial solver is

$$A\,\mathbf{U} \leq \mathbf{b}(k), \tag{28}$$

$$A = \begin{bmatrix} -I_{V_{\text{HOR}} \cdot 4} \\ I_{V_{\text{HOR}} \cdot 4} \\ -\mathcal{M}_m \\ \mathcal{M}_m \end{bmatrix}, \tag{29}$$

$$\mathbf{b}(k) = \begin{bmatrix} -\mathbf{U}_{\min} \\ \mathbf{U}_{\max} \\ \mathbf{Y}_{\text{bar}} - \mathbf{X}_{\min} \\ \mathbf{X}_{\max} - \mathbf{Y}_{\text{bar}} \end{bmatrix}. \tag{30}$$

The constraint matrix A is time-invariant and precomputed offline. We update $\mathbf{b}(k)$ at each time step and use the OSQP to solve the quardratic polynomial. Here we have primal dual tolerances of 10^{-4}, and a maximum of 4000 iterations.

IV. Results

The pipeline was evaluated on five map configurations within the 1000×1000 grid environment. All maps share the same truck endpoints: start $(50,\ 500)$ and end $(950,\ 500)$. The maps vary in delivery point location, number of restricted zones, and obstacle density. Table II summarizes each configuration, where N_T and N_D denote the number of truck and drone restricted zones respectively.

TABLE II
Map Configurations

Map	Delivery point	N_T	N_D
Map 1	(500, 200)	2	2
Map 2	(350, 850)	4	3
Map 3	(700, 100)	2	1
Map 4	(500, 900)	3	3
Map 5	(250, 350)	4	3

All three planners (A*, wind-biased RRT, and standard RRT) were run under identical MPC tracking conditions with the simulation parameters listed in Table III.

TABLE III
Simulation Parameters

Parameter	Value
Grid size N	1000
Cell size Δ_c	1 m
Truck speed v_T	2 m/s
Drone cruise speed V_{cruise}	10 m/s
Wind mean $\bar{\mathbf{w}}$	$[1.6,\ 0.9]^\top$ m/s
OU mean-reversion rate α_w	0.4
OU noise std σ_{OU}	0.3
MPC horizon F	15
Time step T	0.5 s
Accel. bounds $[A_{\min},\ A_{\max}]$	$[-5,\ +5]$ m/s^2
Vel. bounds $[V_{\min},\ V_{\max}]$	$[-12,\ +12]$ m/s
Delivery tolerance ε_D	10 m
Landing tolerance $\varepsilon_{\text{land}}$	15 m
Drone mass m_0	1.0 kg
Mass ratio α	0.75
RRT max iterations I	4000
RRT step size δ	20 cells
Launch search step Δ_s	10

A. Quantitative Comparison

Table IV presents the performance metrics across all five maps. Each planner is evaluated on total path waypoints, drone airtime (time between launch and landing), cumulative mechanical energy, and MPC simulation wall-clock time.

B. Airtime Analysis

A* consistently achieves the shortest airtime across all five maps, with an average of 72.2 s compared to 93.4 s for wind-biased RRT and 91.0 s for standard RRT. This is expected because A* produces geometrically optimal paths on the grid, and the MPC tracker follows them with minimal deviation.

The wind-biased RRT shows an airtime advantage over standard RRT in Maps 2 and 3 (3.5 s and 2.0 s shorter

TABLE IV
PLANNER PERFORMANCE ACROSS FIVE MAPS

Map	Planner	Path (cells)	Airtime (s)	Energy (J)	MPC Sim (ms)	Delivery (s)	Landing (s)
Map 1	A*	766	80.5	839.5	14.6	279.5	322.5
	Wind RRT	56	106.0	1366.4	36.6	286.0	348.0
	Std RRT	54	104.5	1134.8	52.7	290.0	346.5
Map 2	A*	637	66.0	870.3	16.2	139.5	172.0
	Wind RRT	45	82.5	1155.0	36.2	147.0	188.5
	Std RRT	45	86.0	1078.4	51.0	149.5	192.0
Map 3	A*	884	99.5	1055.4	184.4	389.5	441.5
	Wind RRT	59	115.5	1924.1	217.3	401.0	457.5
	Std RRT	61	117.5	2131.6	224.9	409.0	459.5
Map 4	A*	598	63.5	797.6	132.6	266.0	299.5
	Wind RRT	50	100.0	1595.3	229.8	278.5	336.0
	Std RRT	46	84.5	1304.9	162.4	273.5	320.5
Map 5	A*	413	51.5	548.2	128.0	49.5	77.5
	Wind RRT	32	63.0	844.4	150.2	54.0	89.0
	Std RRT	32	62.5	852.5	136.4	51.0	88.5

respectively), where the delivery point creates flight segments aligned with the wind direction $\bar{\mathbf{w}} = [1.6,\ 0.9]^\top$ m/s. In Maps 1 and 5, the two RRT variants perform within 1.5 s of each other, indicating that the wind bias has marginal effect when the flight direction is roughly perpendicular to the wind.

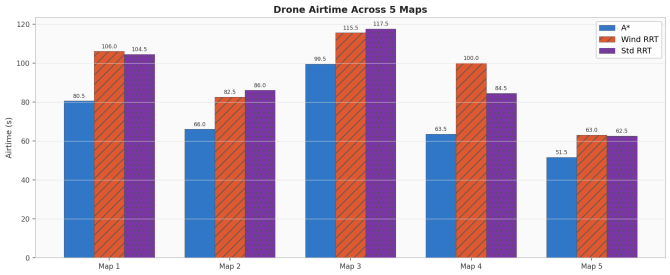

Fig. 2. Drone airtime comparison across five maps.

C. Energy Consumption

A* achieves the lowest energy consumption in every map, averaging 822.2 J versus 1377.0 J for wind-biased RRT and 1300.4 J for standard RRT. The energy gap between A* and the RRT variants is proportionally larger than the airtime gap. This indicates that RRT paths require more control effort per unit time due to sharper directional changes at waypoint transitions, which force the MPC to apply larger corrective accelerations.

Map 3 shows the most pronounced energy difference: A* consumes 1055.4 J while wind-biased RRT consumes 1924.1 J and standard RRT consumes 2131.6 J. This map has the delivery point at $(700,\ 100)$, far below and to the right of the truck road, creating a long outbound leg where suboptimal RRT paths accumulate significant additional energy cost.

D. MPC Execution Time

The MPC simulation wall-clock time scales with the total number of simulation steps, which includes the pre-launch, flight, and post-landing phases. Maps 1 and 2 complete in under 55 ms for all planners, while Maps 3 through 5 require

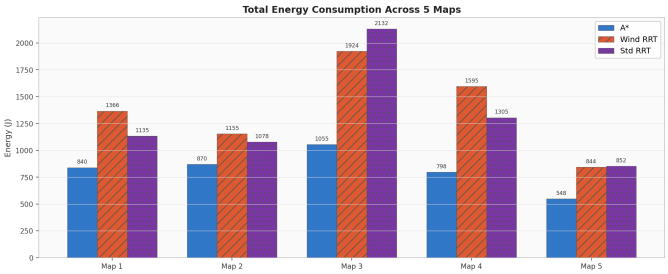

Fig. 3. Total energy consumption across five maps.

128 to 230 ms due to longer truck routes and flight durations. The per-step QP solve is sub-millisecond in all cases.

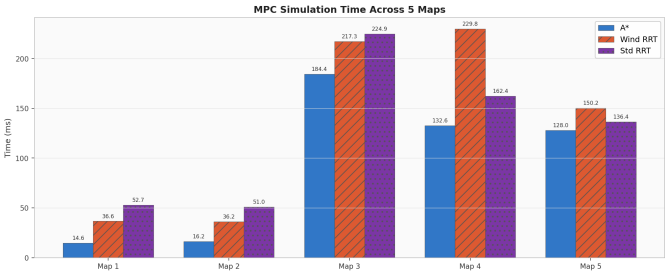

Fig. 4. MPC simulation time across five maps.

E. Time Complexity

Table V summarizes the computational complexity of each pipeline stage. Here R is the number of obstacles, N is the grid side length, $|\Lambda|$ is the number of return multiplier candidates evaluated during the launch search, I is the RRT iteration limit, n is the tree size at a given iteration, F is the MPC prediction horizon, and K_{total} is the total simulation step count.

In the experiments, A* dominates the offline planning cost due to the launch search invoking it $K_{\max}/\Delta_s \approx 90$ times per multiplier value, totaling roughly 450 calls. The RRT variants avoid this repeated cost but incur $O(I \cdot n)$ per path, which

TABLE V
TIME COMPLEXITY OF PIPELINE STAGES

Stage	Complexity
Map creation	$O(R \cdot N^2)$
Truck A*	$O(N^2 \log N)$
Launch search	$O\Big(\frac{K_{\max}}{\Delta_s} \cdot \lvert\Lambda\rvert \cdot N^2 \log N\Big)$
Drone A*	$O(N^2 \log N)$
Standard/Wind RRT	$O(I \cdot n)$
MPC matrix build	$O(F^3)$
MPC per step	$O(F^3)$
Full MPC simulation	$O(K_{\text{total}} \cdot F^3)$

is comparable for $I = 4000$. The MPC online cost remains identical across all planners at $O(F^3)$ per step, confirming that the choice of path planner does not affect real-time controller performance.

V. FUTURE WORK

Several extensions would strengthen the framework:

1) Truck Position Uncertainty: The current model assumes the truck follows its planned path exactly. In practice, traffic, road closures, and driver behavior introduce uncertainty in the truck's position. A probabilistic truck model, for example, modeling the truck position as a Gaussian process along the road arc-length; would allow the drone to plan return paths that are robust to truck delays. The MPC would incorporate the truck position covariance into its cost function, penalizing rendezvous plans that rely on precise truck timing.

2) Drone Localization Uncertainty: The drone's position is currently assumed to be perfectly known. In real deployments, GPS drift, IMU bias, and sensor noise introduce localization error. Extending the MPC to an output-feedback formulation, such as a Kalman filter coupled with the existing QP controller, would account for state estimation uncertainty. The delivery and landing tolerances (ε_D, $\varepsilon_{\text{land}}$) could then be set based on the estimated covariance rather than fixed thresholds.

3) Battery Constraints and Energy-Optimal Planning: The current objective minimizes path length (or equivalently, flight time), but does not impose an energy budget. A practical drone has a fixed battery capacity $E_{\max}$. Incorporating the constraint $E(k_R) \leq E_{\max}$ into the launch search would reject launch candidates where the round trip is energetically infeasible.

4) Learning-Based Wind Models: The Ornstein-Uhlenbeck wind model assumes stationary statistics with known parameters. In real urban environments, wind fields are spatially and temporally non stationary, building wakes, street canyons, and thermal effects create complex flow patterns. A neural network trained on historical wind data (e.g., from weather APIs or onboard anemometers) could provide a spatially varying wind prediction $\hat{\mathbf{w}}(\mathbf{p}, t)$ that replaces the constant $\bar{\mathbf{w}}$ in both the RRT sampling bias and the MPC wind model. Physics-informed neural networks [8] are a promising approach, as they can enforce fluid dynamics constraints while learning from sparse observations.

5) 3D Extension: The current formulation operates in 2D. Real drone delivery requires altitude management, terrain avoidance, building clearance, and altitude-dependent wind profiles. Extending the occupancy grid to 3D ($N \times N \times N_z$), the RRT to 3D sampling, and the MPC to a 6-DOF quadrotor model would bring the framework closer to deployment. The wind-biased sampling generalises naturally to 3D by using the 3D wind vector in the bias direction.

REFERENCES

[1] C. C. Murray and A. G. Chu, "The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery," *Transp. Res. C, Emerg. Technol.*, vol. 54, pp. 86–109, 2015.

[2] M. R. Salama and S. Srinivas, "Collaborative truck multi-drone routing and scheduling problem: Package delivery with flexible launch and recovery sites," *Transp. Res. E*, vol. 164, p. 102788, 2022.

[3] T. Thomas, S. Srinivas, and C. Rajendran, "Collaborative truck multi-drone delivery system considering drone scheduling and en route operations," *Ann. Oper. Res.*, vol. 339, pp. 1–39, 2023.

[4] M. Marinelli, L. Caggiani, M. Ottomanelli, and M. Dell'Orco, "En route truck–drone parcel delivery for optimal vehicle routing strategies," *IET Intell. Transp. Syst.*, vol. 12, no. 4, pp. 253–261, 2018.

[5] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011.

[6] S. Primatesta, A. Osman, and A. Rizzo, "MP-RRT#: A model predictive sampling-based motion planning algorithm for unmanned aircraft systems," *J. Intell. Robot. Syst.*, vol. 103, no. 59, 2021.

[7] G. Thanellas, V. C. Moulianitis, and N. A. Aspragathos, "A spatially wind aware quadcopter (UAV) path planning approach," *IFAC-PapersOnLine*, vol. 52, no. 8, pp. 283–288, 2019.

[8] S. Zhang *et al.*, "A physics-informed neural network approach for UAV path planning in dynamic environments," *arXiv preprint arXiv:2510.21874*, 2025.

[9] M. Kamel, M. Burri, and R. Siegwart, "Linear vs nonlinear MPC for trajectory tracking applied to rotary wing micro aerial vehicles," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 3463–3469, 2017.

[10] A. Paris, B. T. Lopez, and J. P. How, "Dynamic landing of an autonomous quadrotor on a moving platform in turbulent wind conditions," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2020.

[11] Y. Huang, H. Li, Y. Dai, M. Gao, and M. Duan, "A 3D path planning algorithm for UAVs based on an improved artificial potential field and bidirectional RRT*," *Drones*, vol. 8, no. 12, p. 760, 2024.

[12] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[13] D. Mandloi, R. Arya, and A. K. Verma, "Unmanned aerial vehicle path planning based on A* algorithm and its variants in 3D environment," *Int. J. Syst. Assur. Eng. Manag.*, vol. 12, no. 5, pp. 990–1000, 2021.

[14] X. Bai *et al.*, "UAV path planning based on improved A* and DWA algorithms," *Int. J. Aerosp. Eng.*, vol. 2021, Art. no. 4511252, 2021.

[15] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2014, pp. 2997–3004.

[16] D. J. Webb and J. van den Berg, "Kinodynamic RRT*: Asymptotically optimal motion planning for robots with linear dynamics," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2013, pp. 5054–5061.